

5.5 Iterator Pattern

Iterator Pattern เป็นแพตเทิร์นพฤติกรรมที่ใช้สำหรับการเข้าถึงองค์ประกอบของคอลเลกชัน (collection) ที่ละรายการโดยไม่ต้องเปิดเผยโครงสร้างภายในของคอลเลกชัน เหมาะสำหรับการจัดการออบเจกต์หลาย ๆ ตัวที่จัดเก็บในคอลเลกชัน เช่น อุปกรณ์ในบ้านอัจฉริยะ (Smart Home Devices)

:: *Iterator Pattern* ::

เข้าถึงองค์ประกอบของคอลเลกชัน (collection) ที่ละรายการโดยไม่ต้องเปิดเผยโครงสร้างภายใน

เหมาะสำหรับการจัดการออบเจกต์หลาย ๆ ตัวที่จัดเก็บในคอลเลกชัน

ตัวอย่างปัญหา

บ้านอัจฉริยะ มีอุปกรณ์หลายตัว เช่น ไฟในห้องนั่งเล่น เครื่องปรับอากาศในห้องนอน ม่านในห้องครัว ฯลฯ นักพัฒนาซอฟต์แวร์ต้องการควบคุมอุปกรณ์ทั้งหมดโดยไม่ต้องรู้ว่าข้อมูลอุปกรณ์ถูกเก็บในโครงสร้างข้อมูลใด

ตัวอย่าง Main program

```
// ใช้ Iterator เพื่อวนลูปควบคุมอุปกรณ์
Iterator<SmartDevice> iterator =
    smartHome.createIterator();
System.out.println("Turning ON all devices:");
while (iterator.hasNext()) {
    SmartDevice device = iterator.next();
    device.turnOn();
}
```

5.5.1 วัตถุประสงค์และตัวอย่าง

Iterator pattern ช่วยให้เข้าถึงองค์ประกอบต่าง ๆ ของอ็อบเจกต์รวม (Aggregate Object) ตามลำดับ โดยไม่ต้องเปิดเผยโครงสร้างภายในของอ็อบเจกต์นั้น ช่วยแก้ปัญหาหลักได้แก่

1) การแยกส่วนการสำรวจ (Decoupling Traversal Logic)

ถ้ามีโครงสร้างข้อมูลหลายแบบ เช่น Array, List, Tree, Hash Map การวนซ้ำ (Traversal) องค์ประกอบแต่ละแบบจะต้องเขียนโค้ดที่แตกต่างกัน แก้ไขโดยใช้ Iterator เพื่อแยกหน้าที่การวนซ้ำออกเป็นอ็อบเจกต์ตัววนซ้ำ (Iterator Object) โดยเฉพาะ ทำให้ Context (Client) ที่ต้องการวนซ้ำไม่ต้องรู้ว่าโครงสร้างภายในของคอลเลกชันคืออะไร จะรู้เพียงแค่ต้องเรียกเมธอดมาตรฐาน เช่น next(), hasNext()

2) การทำให้โครงสร้างภายในเป็นส่วนตัว (Encapsulating Internal Structure)

ถ้าต้องการให้ Client เข้าถึงข้อมูลในคอลเลกชัน อาจต้องให้ Client รู้จักวิธีการทำงานภายในของคอลเลกชันนั้น ๆ เช่น การรู้ว่าต้องวน for อย่างไร หรือต้องใช้ index อย่างไร ทำให้เกิด Tight Coupling ดังนั้น คอลเลกชัน (Aggregate Object) จึงมีเมธอดที่สร้าง Iterator ออกมาให้ เพื่อทำหน้าที่เป็นสะพานในการเข้าถึงข้อมูล และ Client ติดต่อกับ Iterator ผ่าน Interface มาตรฐาน ทำให้โครงสร้างภายใน เช่น เป็น Array, Linked List หรือ Tree ยังคงเป็นความลับและสามารถเปลี่ยนแปลงได้โดยไม่กระทบต่อ Client

3) การรองรับการวนซ้ำหลายแบบ (Supporting Multiple Traversals)

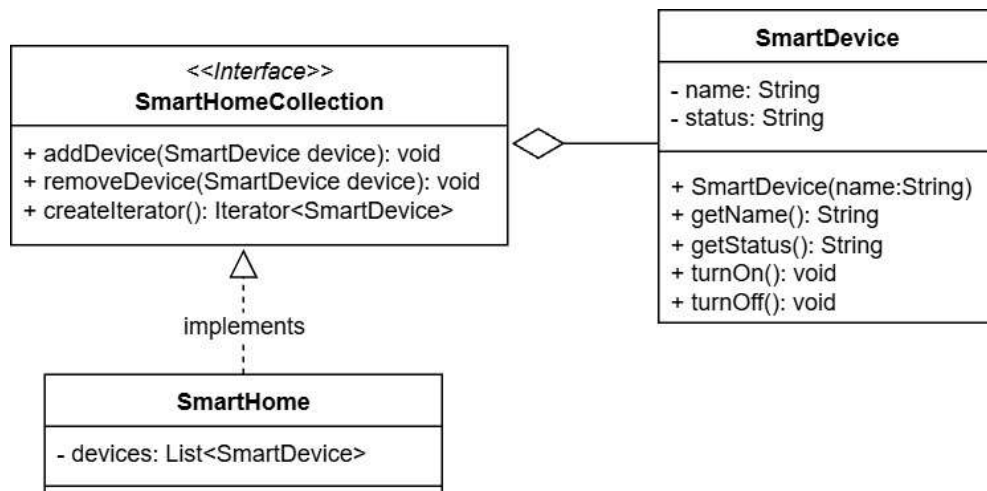
วัตถุประสงค์ย่อยคือช่วยให้สามารถมีหลายตัววนซ้ำ ที่ทำงานพร้อมกันบนคอลเลกชันเดียวกันได้ และยังสามารถรองรับรูปแบบการวนซ้ำที่แตกต่างกัน เช่น การวนซ้ำตามลำดับปกติ การวนซ้ำย้อนกลับ โดยไม่ต้องแก้ไขคลาสคอลเลกชันหลัก

ในบริบทของ "บ้านอัจฉริยะ" (Smart Home) อาจมีสถานการณ์ดังนี้

- การจัดการอุปกรณ์หลายชนิด บ้านอาจมีอุปกรณ์หลายประเภท เช่น ไฟ เครื่องปรับอากาศ และม่านหน้าต่าง ซึ่งต้องการเข้าถึงหรือควบคุมทีละตัว
- การสนับสนุนหลายรูปแบบของคอลเลกชัน อุปกรณ์อาจจัดเก็บในรูปแบบต่าง ๆ เช่น Array, List, หรือ Map
- การซ่อนรายละเอียดของโครงสร้างข้อมูล ผู้ใช้งานไม่จำเป็นต้องรู้ว่าอุปกรณ์ถูกจัดเก็บอย่างไร แต่สามารถวนลูปเพื่อควบคุมได้

ตัวอย่างการใช้งาน

สมมติว่าในบ้านอัจฉริยะ มีอุปกรณ์หลายตัว เช่น ไฟในห้องนั่งเล่น เครื่องปรับอากาศในห้องนอน ม่านในห้องครัว ฯลฯ นักพัฒนาซอฟต์แวร์ต้องการควบคุมอุปกรณ์ทั้งหมดโดยไม่ต้องรู้ว่าข้อมูลอุปกรณ์ถูกเก็บในโครงสร้างข้อมูลใด



ภาพที่ xx ระบบจัดการบ้านอัจฉริยะที่มีอุปกรณ์หลายตัวโดยใช้ Iterator Pattern

SmartDevice

- แสดงถึงอุปกรณ์แต่ละตัวในบ้าน เช่น ไฟหรือเครื่องปรับอากาศ
- มีสถานะ (ON หรือ OFF) และเมธอดสำหรับควบคุม

```

1. // SmartDevice Class: แทนอุปกรณ์ในบ้าน
2. class SmartDevice {
3.
4.     private String name;
5.     private String status;
6.
7.     public SmartDevice(String name) {
8.         this.name = name;
9.         this.status = "OFF";
10.    }
11.
12.    public String getName() {
13.        return name;
14.    }
15.
16.    public String getStatus() {
17.        return status;
18.    }
19.
20.    public void turnOn() {
21.        status = "ON";
22.        System.out.println(name + " is now ON.");
23.    }
24.
25.    public void turnOff() {
26.        status = "OFF";
27.        System.out.println(name + " is now OFF.");
  
```

```
28. }
29. }
```

SmartHomeCollection Interface

1. กำหนดโครงสร้างของคอลเลกชันที่เก็บอุปกรณ์ในบ้าน
2. มีเมธอด createIterator สำหรับสร้าง Iterator

```
1. // SmartHomeCollection Interface: กำหนดคอลเลกชันของอุปกรณ์
2. interface SmartHomeCollection {
3.     void addDevice(SmartDevice device);
4.     void removeDevice(SmartDevice device);
5.     Iterator<SmartDevice> createIterator();
6. }
```

SmartHome

1. คอลเลกชันที่ใช้เก็บอุปกรณ์ในรูปแบบ List
2. ใช้ createIterator เพื่อสร้าง Iterator สำหรับวนลูป

```
1. // Concrete Collection: เก็บอุปกรณ์ในบ้าน
2. class SmartHome implements SmartHomeCollection {
3.
4.     private List<SmartDevice> devices = new ArrayList<>();
5.
6.     @Override
7.     public void addDevice(SmartDevice device) {
8.         devices.add(device);
9.     }
10.
11.    @Override
12.    public void removeDevice(SmartDevice device) {
13.        devices.remove(device);
14.    }
15.
16.    @Override
17.    public Iterator<SmartDevice> createIterator() {
18.        return devices.iterator();
19.    }
20. }
```

Client หรือ Main Class ใช้ Iterator เพื่อวนลูปควบคุมอุปกรณ์ในบ้านโดยไม่ต้องรู้โครงสร้างภายในของคอลเลกชัน

```
1. // Main Class
2. public class SmartHomeIterator {
3.     public static void main(String[] args) {
4.
```

```

5.         // สร้างคอลเลกชันของบ้าน
6.         SmartHome smartHome = new SmartHome();
7.
8.         // เพิ่มอุปกรณ์ในบ้าน
9.         SmartDevice livingRoomLight = new
10.             SmartDevice("Living Room Light");
11.         SmartDevice bedroomAC = new
12.             SmartDevice("Bedroom Air Conditioner");
13.         SmartDevice kitchenCurtains = new
14.             SmartDevice("Kitchen Curtains");
15.
16.         smartHome.addDevice(livingRoomLight);
17.         smartHome.addDevice(bedroomAC);
18.         smartHome.addDevice(kitchenCurtains);
19.
20.         // ใช้ Iterator เพื่อวนลูปควบคุมอุปกรณ์
21.         Iterator<SmartDevice> iterator =
22.             smartHome.createIterator();
23.         System.out.println("Turning ON all devices:");
24.         while (iterator.hasNext()) {
25.             SmartDevice device = iterator.next();
26.             device.turnOn();
27.         }
28.
29.         System.out.println("\nTurning OFF all devices:");
30.         // สร้าง iterator ใหม่
31.         iterator = smartHome.createIterator();
32.         while (iterator.hasNext()) {
33.             SmartDevice device = iterator.next();
34.             device.turnOff();
35.         }
36.     }
37. }

```

ผลลัพธ์

Turning ON all devices:

Living Room Light is now ON.

Bedroom Air Conditioner is now ON.

Kitchen Curtains is now ON.

Turning OFF all devices:

Living Room Light is now OFF.

Bedroom Air Conditioner is now OFF.

Kitchen Curtains is now OFF.

ข้อดีของการใช้ Iterator Pattern ในระบบบ้านอัจฉริยะ

- แยกความรับผิดชอบ คอลเลกชัน (SmartHome) ไม่จำเป็นต้องจัดการการวนลูปโดยตรง
- ยืดหยุ่น สามารถเปลี่ยนโครงสร้างข้อมูลของคอลเลกชัน เช่น จาก List เป็น Set โดยไม่กระทบกับโค้ดที่ใช้ Iterator
- ใช้งานง่าย ลดความซับซ้อนในการเข้าถึงอุปกรณ์ทั้งหมด
- รองรับการขยาย สามารถเพิ่มประเภทคอลเลกชันใหม่ เช่น คอลเลกชันอุปกรณ์ IoT ในอนาคต

กรณีการขยาย

- เพิ่มฟีเจอร์ใหม่ เพื่อควบคุมเฉพาะอุปกรณ์บางประเภท เช่น ไฟ ม่าน หรือเครื่องปรับอากาศ
- รองรับหลายห้อง เพิ่มการจัดกลุ่มอุปกรณ์ตามห้อง เช่น ห้องนั่งเล่นหรือห้องนอน
- สร้าง Iterator แบบเฉพาะเจาะจง เช่น Iterator ที่เลือกเฉพาะอุปกรณ์ที่อยู่ในสถานะ OFF

5.5.2 ตัวอย่างการประยุกต์ใช้ในการทำงานจริง

Iterator Pattern ใช้สำหรับการเข้าถึงและวนลูปผ่านคอลเลกชันของออบเจกต์ โดยไม่ต้องเปิดเผยโครงสร้างภายในของคอลเลกชัน ตัวอย่างปัญหาจากการใช้งานจริงที่เหมาะสมกับ Iterator Pattern มีดังนี้

ตัวอย่างที่ 1 ระบบเมนูร้านอาหาร (Restaurant Menu System) ระบบเมนูร้านอาหารสามารถใช้ Iterator Pattern เพื่อจัดการเมนูหลายประเภท เช่น อาหารคาว อาหารหวาน และเครื่องดื่ม โดยไม่ต้องเปิดเผยโครงสร้างข้อมูลภายในของเมนูแต่ละประเภท ซึ่งจัดเก็บในคอลเลกชันที่แตกต่างกัน

- อาหารคาวใน Array
- อาหารหวานใน List
- เครื่องดื่มใน Set

ผู้ต้องการเข้าถึงเมนูทั้งหมดโดยไม่ต้องสนใจโครงสร้างข้อมูล และวนลูปเฉพาะเมนูที่มีราคาไม่เกิน 100 บาท

การใช้ Iterator Pattern

- สร้าง Iterator สำหรับคอลเลกชันเมนูแต่ละประเภท
- สร้าง Composite Iterator เพื่อรวมเมนูจากทุกประเภทให้เป็นลำดับเดียวกัน

โครงสร้างระบบ

- MenuItem Class ตัวแทนของแต่ละรายการในเมนู
- Menu Interface กำหนดโครงสร้างของเมนู เช่น การเพิ่ม/ลบรายการ และการสร้าง Iterator
- Concrete Menus เมนูสำหรับประเภทอาหาร เช่น อาหารคาว อาหารหวาน เครื่องดื่ม
- Iterator Interface ใช้สำหรับวนลูปเมนู
- Main Class ใช้แสดงและจัดการเมนู

MenuItem ตัวแทนรายการอาหารแต่ละรายการในเมนู เก็บข้อมูลชื่อ, รายละเอียด, และราคา

```
1. // MenuItem Class: แทนรายการในเมนู
2. class MenuItem {
3.
4.     private String name;
5.     private String description;
6.     private double price;
7.
8.     public MenuItem(String name, String description
9.                     , double price) {
10.         this.name = name;
11.         this.description = description;
12.         this.price = price;
13.     }
14.
15.     public String getName() {
16.         return name;
17.     }
18.
19.     public String getDescription() {
20.         return description;
21.     }
22.
23.     public double getPrice() {
24.         return price;
25.     }
26.
27.     @Override
28.     public String toString() {
29.         return name + " (" + description + ") - $" + price;
30.     }
31. }
```

Menu Interface กำหนดโครงสร้างพื้นฐานของเมนู เช่น การเพิ่มรายการ (addItem) และการสร้าง Iterator (createIterator)

```
1. // Menu Interface: กำหนดโครงสร้างเมนู
2. interface Menu {
3.
```

```

4.     void addItem(String name, String description
5.         , double price);
6.     Iterator<MenuItem> createIterator();
7. }

```

Concrete Menus MainCourseMenu: เก็บข้อมูลใน ArrayList

```

1. // Concrete Menu: สำหรับอาหารคาว
2. class MainCourseMenu implements Menu {
3.
4.     private List<MenuItem> menuItems;
5.
6.     public MainCourseMenu() {
7.         menuItems = new ArrayList<>();
8.         addItem("Steak", "Grilled steak with sauce", 15.99);
9.         addItem("Pasta", "Creamy Alfredo pasta", 12.49);
10.    }
11.
12.    @Override
13.    public void addItem(String name, String description
14.        , double price) {
15.        menuItems.add(new MenuItem(name, description, price));
16.    }
17.
18.    @Override
19.    public Iterator<MenuItem> createIterator() {
20.        return menuItems.iterator();
21.    }
22.}

```

DessertMenu เก็บข้อมูลใน Array และใช้ ArrayIterator เพื่อวนลูป

```

1. // Concrete Menu: สำหรับอาหารหวาน
2. class DessertMenu implements Menu {
3.
4.     private MenuItem[] menuItems;
5.     private int numberOfItems = 0;
6.
7.     public DessertMenu() {
8.         // กำหนดขนาด Array
9.         menuItems = new MenuItem[5];
10.        addItem("Cheesecake", "Classic New York cheesecake"
11.            , 6.99);
12.        addItem("Ice Cream", "Vanilla ice cream with chocolate
13.            sauce", 4.49);
14.    }
15.
16.    @Override
17.    public void addItem(String name, String description
18.        , double price) {
19.        if (numberOfItems >= menuItems.length) {
20.            System.out.println(
21.                "Menu is full! Cannot add more items.");
22.            return;
23.        }
24.        menuItems[numberOfItems] = new

```

```

25.             MenuItem(name, description, price);
26.         numberOfItems++;
27.     }
28.
29.     @Override
30.     public Iterator<MenuItem> createIterator() {
31.         return new ArrayIterator(menuItems);
32.     }
33.}

```

Iterator ใช้สำหรับเข้าถึงรายการในเมนูโดยไม่ต้องเปิดเผยโครงสร้างภายใน

```

1. // Array Iterator: สำหรับเมนูที่ใช้ Array
2. class ArrayIterator implements Iterator<MenuItem> {
3.
4.     private MenuItem[] items;
5.     private int position = 0;
6.
7.     public ArrayIterator(MenuItem[] items) {
8.         this.items = items;
9.     }
10.
11.     @Override
12.     public boolean hasNext() {
13.         return position < items.length && items[position]
14.             != null;
15.     }
16.
17.     @Override
18.     public MenuItem next() {
19.         return items[position++];
20.     }
21.}

```

Main Class สร้างเมนูและแสดงรายการอาหารทั้งหมดโดยใช้ Iterator

```

1. // Main Class: ใช้งานระบบเมนู
2. public class RestaurantMenuSystem {
3.     public static void main(String[] args) {
4.
5.         // สร้างเมนูแต่ละประเภท
6.         Menu mainCourseMenu = new MainCourseMenu();
7.         Menu dessertMenu = new DessertMenu();
8.
9.         // แสดงรายการเมนูทั้งหมด
10.        System.out.println("Main Course Menu:");
11.        printMenu(mainCourseMenu.createIterator());
12.
13.        System.out.println("\nDessert Menu:");
14.        printMenu(dessertMenu.createIterator());
15.    }
16.
17.    public static void printMenu(Iterator<MenuItem> iterator) {
18.        while (iterator.hasNext()) {
19.            MenuItem menuItem = iterator.next();
20.            System.out.println(menuItem);

```

21. }
22. }
23. }

ผลลัพธ์

Main Course Menu:

Steak (Grilled steak with sauce) - \$15.99

Pasta (Creamy Alfredo pasta) - \$12.49

Dessert Menu:

Cheesecake (Classic New York cheesecake) - \$6.99

Ice Cream (Vanilla ice cream with chocolate sauce) - \$4.49

ตัวอย่างที่ 2 ระบบจัดการสินค้าในคลังสินค้า (Warehouse Management System) ในคลัง
สินค้ามีสินค้าหลายหมวดหมู่ เช่น อุปกรณ์อิเล็กทรอนิกส์ เสื้อผ้า และอาหาร ถูกจัดเก็บในโครงสร้าง
ข้อมูลแบบต่าง ๆ ได้แก่

- อุปกรณ์อิเล็กทรอนิกส์ใน Map ได้แก่ชุดข้อมูลของชื่อสินค้าและจำนวน
- เสื้อผ้าใน List
- อาหารใน Array

ผู้ใช้งานต้องการ

- ตรวจสอบสินค้าทั้งหมดโดยไม่สนใจโครงสร้างข้อมูล
- กรองสินค้าที่มีจำนวนต่ำกว่าค่าที่กำหนด

การใช้งาน Iterator Pattern

- ใช้ Iterator เพื่อวนลูปสินค้าทุกประเภท
- เพิ่มฟังก์ชันการกรองสำหรับสินค้าที่มีจำนวนต่ำ

ตัวอย่างที่ 3 ระบบแสดงรายการในแอปพลิเคชันสตรีมมิ่ง (Streaming Application) แอปพลิเคชัน
สตรีมมิ่งมีรายการเนื้อหาหลายประเภท เช่น ภาพยนตร์ ซีรีส์ และสารคดี ซึ่งจัดเก็บแยกกัน

- ภาพยนตร์ใน Array
- ซีรีส์ใน List
- สารคดีใน HashMap ชุดข้อมูลของชื่อและผู้สร้างสารคดี

ผู้ใช้งานต้องการ

- วนลูปเนื้อหาทั้งหมดเพื่อสร้างหน้าหลัก
- กรองเฉพาะเนื้อหาที่มีคะแนนความนิยมสูง

การใช้ Iterator Pattern

- สร้าง Iterator สำหรับแต่ละประเภทของเนื้อหา
- สร้าง Iterator รวมสำหรับจัดเรียงเนื้อหาทั้งหมดในลำดับเดียว

ตัวอย่างที่ 4 ระบบสร้างรายงานพนักงาน (Employee Report System) ในองค์กรมีพนักงานที่จัดเก็บในโครงสร้างข้อมูลต่าง ๆ เช่น

- พนักงานประจำใน List
- พนักงานชั่วคราวใน Array
- พนักงานภายนอกใน Set

ฝ่ายทรัพยากรบุคคลต้องการ

- ดึงข้อมูลพนักงานทั้งหมดเพื่อสร้างรายงาน
- กรองเฉพาะพนักงานที่อยู่ในสถานะ "Active"

การใช้ Iterator Pattern

- ใช้ Iterator เพื่อวนลูปพนักงานทุกประเภท
- เพิ่มฟังก์ชันการกรองพนักงานที่สถานะเป็น "Active"

ตัวอย่างที่ 5 ระบบแสดงไฟล์ในโปรแกรมจัดการไฟล์ (File Explorer)

ในโปรแกรมจัดการไฟล์ มีโพลเดอร์ที่เก็บไฟล์หลายประเภท เช่น

- เอกสารใน List
- ไฟล์รูปภาพใน Set
- ไฟล์วิดีโอใน Array

ผู้ใช้งานต้องการ

- เข้าถึงไฟล์ทั้งหมดในลำดับเดียว
- กรองเฉพาะไฟล์ที่มีขนาดใหญ่กว่า 100MB

การใช้ Iterator Pattern

- สร้าง Iterator สำหรับไฟล์ในแต่ละประเภท
- ใช้ Composite Iterator เพื่อรวมไฟล์ทั้งหมดในลำดับเดียว

ตัวอย่างที่ 6 ระบบแสดงผลการค้นหาในเว็บไซต์อีคอมเมิร์ซ (E-commerce Search Results)

ในเว็บไซต์อีคอมเมิร์ซ ผลการค้นหาสินค้าถูกดึงมาจากแหล่งข้อมูลหลายแหล่ง เช่น

- สินค้าในคลังสินค้า (List)
- สินค้าจากพาร์ทเนอร์ (Set)
- สินค้าลดราคา (Array)

ผู้ใช้ต้องการ

- วนลูปผลการค้นหาทั้งหมดเพื่อแสดงบนหน้าเว็บไซต์
- จัดเรียงผลการค้นหาโดยคะแนนความนิยม

การใช้ Iterator Pattern

- ใช้ Iterator เพื่อเข้าถึงสินค้าจากแต่ละแหล่งข้อมูล
- รวมผลการค้นหาทั้งหมดในลำดับเดียว

ตัวอย่างที่ 7 ระบบแสดงข้อความในโซเชียลมีเดีย (Social Media Feed) ในแอปพลิเคชันโซเชียลมีเดีย

ข้อความถูกดึงมาจากแหล่งข้อมูลต่าง ๆ เช่น

- โพสต์ของเพื่อนใน Array
- โพสต์จากกลุ่มใน List
- โฆษณาใน Set

ผู้ใช้ต้องการ

- วนลูปข้อความทั้งหมดเพื่อแสดงในฟีดเดียว
- กรองเฉพาะโพสต์ที่มีจำนวนไลก์สูง

การใช้ Iterator Pattern

- ใช้ Iterator เพื่อรวมข้อความจากหลายแหล่งข้อมูล
- เพิ่มฟังก์ชันการกรองตามจำนวนไลก์

ตัวอย่างที่ 8 ระบบตรวจสอบเงื่อนไขในไฟล์ JSON หรือ XML

ในระบบที่วิเคราะห์ไฟล์ JSON หรือ XML อาจมีหลายโหนดข้อมูล เช่น

- โหนดลูกค้า
- โหนดคำสั่งซื้อ
- โหนดสินค้า

ผู้ใช้งานต้องการ

- วนลูปโหนดทั้งหมดเพื่อแสดงผล
- กรองเฉพาะโหนดที่มีสถานะ "Active"

การใช้ Iterator Pattern

- ใช้ Iterator สำหรับโหนดในไฟล์ JSON หรือ XML
- เพิ่มฟังก์ชันการกรองสำหรับโหนดที่ตรงตามเงื่อนไข

5.6 State Pattern

State Pattern เป็นแพตเทิร์นพฤติกรรมที่ใช้ในการจัดการพฤติกรรมของออบเจกต์ที่เปลี่ยนไปตามสถานะของตัวเอง โดยแยกสถานะออกจากออบเจกต์หลัก ทำให้การเพิ่มหรือแก้ไขพฤติกรรมในแต่ละสถานะทำได้ง่าย ทำให้ลดการใช้เงื่อนไข if-else หรือ switch โดยใช้แนวคิดของ Polymorphism ในการจัดการพฤติกรรมของแต่ละสถานะ

:: *State Pattern* ::

จัดการพฤติกรรมของออบเจกต์ที่เปลี่ยนไปตามสถานะของตัวเอง

โดยแยกสถานะออกจากออบเจกต์หลัก ทำให้การเพิ่มหรือแก้ไขพฤติกรรมในแต่ละสถานะทำได้ง่าย ทำให้ลดการใช้เงื่อนไข if-else

ตัวอย่างปัญหา

ระบบบ้านอัจฉริยะที่มีไฟในบ้านสามารถทำงานในหลายโหมด เช่น on, off, dim ต้องการควบคุมการเปลี่ยนสถานะด้วยคำสั่งเดียวคือ pressSwitch()

ตัวอย่าง Main program

```
Light light = new Light();  
light.pressSwitch(); // OFF -> ON
```

```
light.pressSwitch(); // ON -> DIM  
light.pressSwitch(); // DIM -> OFF  
light.pressSwitch(); // OFF -> ON
```

5.6.1 วัตถุประสงค์และตัวอย่าง

วัตถุประสงค์หลักของ State Pattern คือการอนุญาตให้อ็อบเจกต์สามารถเปลี่ยนแปลงพฤติกรรมของตัวเองได้เมื่อสถานะภายใน (internal state) เปลี่ยนไป โดยที่อ็อบเจกต์ภายนอกจะรับทราบว่าอ็อบเจกต์นั้นได้เปลี่ยนสถานะไปแล้ว

1) การกำจัดโครงสร้างเงื่อนไขที่ซับซ้อน (Eliminate Complex Conditionals)

เมื่ออ็อบเจกต์มีหลายสถานะ และแต่ละสถานะมีพฤติกรรมแตกต่างกัน ใน 1 เมธอดเดียว อาจประกอบไปด้วยการตรวจสอบเงื่อนไขด้วย switch หรือ if-else ที่ซับซ้อนจำนวนมาก จึงควรใช้ State Pattern เพื่อแยกพฤติกรรมที่เกี่ยวข้องกับแต่ละสถานะออกมาเป็นคลาสใหม่ (State Classes) โดยเฉพาะ

2) การทำให้พฤติกรรมเปลี่ยนแปลงตามสถานะ (Behavior Delegation) เพื่อให้อ็อบเจกต์หลัก (Context) มอบหมาย (Delegate) การจัดการคำขอให้กับอ็อบเจกต์สถานะปัจจุบัน

3) การเปลี่ยนสถานะอย่างเป็นระบบ (Systematic State Transitions) เพื่อควบคุมการเปลี่ยนจากสถานะหนึ่งไปยังอีกสถานะหนึ่งให้อยู่ในความรับผิดชอบของคลาสสถานะเอง คลาสสถานะแต่ละคลาสจะกำหนดตรรกะสำหรับการเปลี่ยนไปยังสถานะถัดไป (Transition) ทำให้การจัดการการเปลี่ยนสถานะมีความชัดเจนและเป็นระเบียบ

ตัวอย่างการใช้งาน

ระบบบ้านอัจฉริยะที่มีไฟในบ้านสามารถทำงานในหลายโหมด เช่น

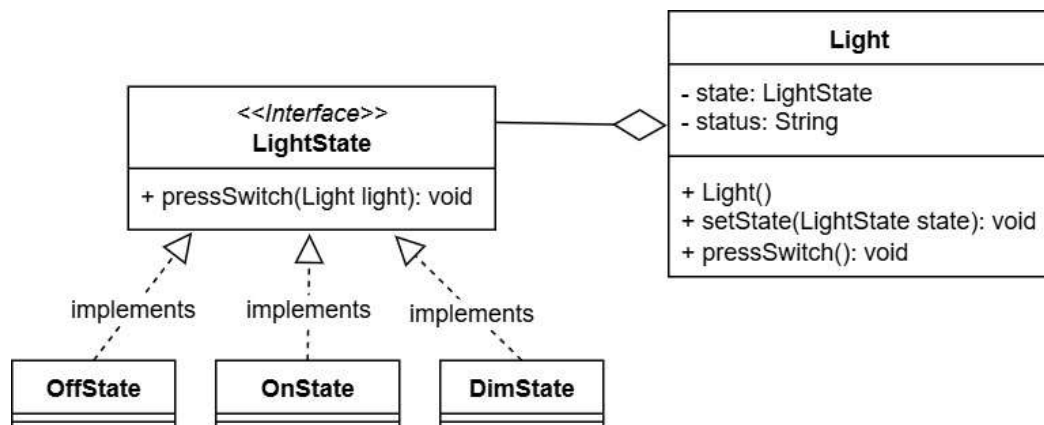
- Off Mode หมายถึง ไฟปิดอยู่
- On Mode หมายถึง ไฟเปิด
- Dim Mode หมายถึง ไฟหรี่แสงลง

การเปลี่ยนแปลงสถานะของไฟจะมีผลต่อพฤติกรรมของไฟ เช่น

- จาก Off → On
- จาก On → Dim
- จาก Dim → Off

โครงสร้างระบบควบคุมไฟในบ้าน

- State Interface กำหนดพฤติกรรมที่ทุกสถานะต้องมี
- Concrete States แต่ละสถานะของไฟ เช่น OffState, OnState, DimState
- Context (Light) ออบเจกต์หลักที่สถานะของมันเปลี่ยนแปลงได้



ภาพที่ xx ระบบบ้านอัจฉริยะที่มีไฟในบ้านสามารถทำงานในหลายโหมด โดยใช้ State Pattern

State Interface (`LightState`) กำหนดเมธอด `pressSwitch()` ให้ทุกสถานะต้องมี

```
1. // State interface
2. interface LightState {
3.     void pressSwitch(Light light);
4. }
```

Concrete States (`OffState`, `OnState`, `DimState`) แต่ละสถานะมีพฤติกรรมเฉพาะเมื่อสวิตช์ถูกกด เช่น

- `OffState` เปลี่ยนเป็น `OnState`
- `OnState` เปลี่ยนเป็น `DimState`
- `DimState` เปลี่ยนเป็น `OffState`

```

1. // Concrete State: Off State
2. class OffState implements LightState {
3.
4.     @Override
5.     public void pressSwitch(Light light) {
6.         System.out.println("Light is now ON.");
7.         light.setState(new OnState());
8.     }
9. }

```

```

1. // Concrete State: On State
2. class OnState implements LightState {
3.
4.     @Override
5.     public void pressSwitch(Light light) {
6.         System.out.println("Light is now in DIM mode.");
7.         light.setState(new DimState());
8.     }
9. }

```

```

1. // Concrete State: Dim State
2. class DimState implements LightState {
3.
4.     @Override
5.     public void pressSwitch(Light light) {
6.         System.out.println("Light is now OFF.");
7.         light.setState(new OffState());
8.     }
9. }

```

Context (Light) มีสถานะปัจจุบัน (state) และเมธอด `pressSwitch()` ที่เปลี่ยนสถานะโดยการเรียกพฤติกรรมของสถานะปัจจุบัน

```

1. // Context: Light
2. class Light {
3.
4.     private LightState state;
5.
6.     public Light() {
7.         // Initial state is Off
8.         state = new OffState();
9.     }
10.
11.    public void setState(LightState state) {
12.        this.state = state;
13.    }
14.
15.    public void pressSwitch() {
16.        state.pressSwitch(this);
17.    }
18.}

```

ตัวอย่างการใช้งาน

```
1. // Main class
2. public class StatePatternExample {
3.     public static void main(String[] args) {
4.
5.         Light light = new Light();
6.
7.         // Simulate pressing the switch
8.         light.pressSwitch(); // OFF -> ON
9.         light.pressSwitch(); // ON -> DIM
10.        light.pressSwitch(); // DIM -> OFF
11.        light.pressSwitch(); // OFF -> ON
12.    }
13.}
```

ผลลัพธ์

Light is now ON.

Light is now in DIM mode.

Light is now OFF.

Light is now ON.

ความจำเป็นในการใช้ State Pattern

- ลดเงื่อนไขซับซ้อน โค้ดไม่ต้องใช้ if-else หรือ switch เพื่อจัดการสถานะ
- ขยายได้ง่าย สามารถเพิ่มสถานะใหม่ เช่น "Night Mode" โดยไม่กระทบโค้ดอื่น
- แยกหน้าที่ชัดเจน พฤติกรรมในแต่ละสถานะถูกแยกออกเป็นคลาสเฉพาะ ทำให้ง่ายต่อการจัดการ

State Pattern เหมาะสำหรับระบบบ้านอัจฉริยะที่พฤติกรรมของอุปกรณ์เปลี่ยนไปตามสถานะ เช่น ไฟ เครื่องปรับอากาศ หรือประตูลิฟต์

5.6.2 ตัวอย่างการประยุกต์ใช้ในการทำงานจริง

State Pattern ทำให้ได้ง่ายต่อการขยายและลดการใช้เงื่อนไขซับซ้อน เช่น if-else หรือ switch-case

ตัวอย่างที่ 1 ระบบควบคุมประตูอัตโนมัติ (Automatic Door Control System) ในระบบประตูอัตโนมัติ ประตูสามารถมีสถานะได้หลายแบบ ได้แก่

- Closed หมายถึง ประตูปิด
- Opening หมายถึง ประตูกำลังเปิด
- Opened หมายถึง ประตูเปิด
- Closing หมายถึง ประตูกำลังปิด

```

1. // State interface
2. interface DoorState {
3.     void handle(Door door);
4. }

```

```

1. // Concrete State: Closed
2. class ClosedState implements DoorState {
3.
4.     @Override
5.     public void handle(Door door) {
6.         System.out.println("Door is opening...");
7.         door.setState(new OpeningState());
8.     }
9. }

```

```

1. // Concrete State: Opening
2. class OpeningState implements DoorState {
3.
4.     @Override
5.     public void handle(Door door) {
6.         System.out.println("Door is now opened.");
7.         door.setState(new OpenedState());
8.     }
9. }

```

```

1. // Concrete State: Opened
2. class OpenedState implements DoorState {
3.
4.     @Override
5.     public void handle(Door door) {
6.         System.out.println("Door is closing...");
7.         door.setState(new ClosingState());
8.     }
9. }

```

```

1. // Concrete State: Closing
2. class ClosingState implements DoorState {
3.
4.     @Override

```

```

5.     public void handle(Door door) {
6.         System.out.println("Door is now closed.");
7.         door.setState(new ClosedState());
8.     }
9. }

```

```

1. // Context: Door
2. class Door {
3.
4.     private DoorState state;
5.
6.     public Door() {
7.         state = new ClosedState(); // Initial state
8.     }
9.
10. public void setState(DoorState state) {
11.     this.state = state;
12. }
13.
14. public void pressButton() {
15.     state.handle(this);
16. }
17.}

```

```

1. // Main Class
2. public class DoorStateExample {
3.     public static void main(String[] args) {
4.
5.         Door door = new Door();
6.
7.         door.pressButton(); // Closed -> Opening
8.         door.pressButton(); // Opening -> Opened
9.         door.pressButton(); // Opened -> Closing
10.        door.pressButton(); // Closing -> Closed
11.    }
12.}

```

ผลลัพธ์

Door is opening...

Door is now opened.

Door is closing...

Door is now closed.

ตัวอย่างที่ 2 ระบบเครื่องขายตัวอัตโนมัติ (Vending Machine) ในระบบเครื่องขายตัวอัตโนมัติ มีสถานะดังนี้

- Idle หมายถึง รอการใส่เงิน
- Processing หมายถึง กำลังประมวลผลคำสั่งซื้อ
- Dispensing หมายถึง จ่ายสินค้า
- Finished หมายถึง เสร็จสิ้นและพร้อมสำหรับการทำงานใหม่

```

1. // State interface
2. interface VendingMachineState {
3.     void handle(VendingMachine machine);
4. }

```

```

1. // Concrete State: Idle
2. class IdleState implements VendingMachineState {
3.
4.     @Override
5.     public void handle(VendingMachine machine) {
6.         System.out.println("Money inserted. Processing...");
7.         machine.setState(new ProcessingState());
8.     }
9. }

```

```

1. // Concrete State: Processing
2. class ProcessingState implements VendingMachineState {
3.
4.     @Override
5.     public void handle(VendingMachine machine) {
6.         System.out.println("Dispensing item...");
7.         machine.setState(new DispensingState());
8.     }
9. }

```

```

1. // Concrete State: Dispensing
2. class DispensingState implements VendingMachineState {
3.
4.     @Override
5.     public void handle(VendingMachine machine) {
6.         System.out.println("Item dispensed. Thank you!");
7.         machine.setState(new FinishedState());
8.     }
9. }

```

```

1. // Concrete State: Finished
2. class FinishedState implements VendingMachineState {
3.
4.     @Override
5.     public void handle(VendingMachine machine) {
6.         System.out.println("Ready for next customer.");
7.         machine.setState(new IdleState());
8.     }
9. }

```

```

1. // Context: VendingMachine
2. class VendingMachine {
3.
4.     private VendingMachineState state;
5.
6.     public VendingMachine() {
7.         state = new IdleState(); // Initial state
8.     }
9.
10. public void setState(VendingMachineState state) {
11.     this.state = state;
12. }
13.
14. public void pressButton() {
15.     state.handle(this);
16. }
17.}

1. // Main Class
2. public class VendingMachineExample {
3.     public static void main(String[] args) {
4.
5.         VendingMachine machine = new VendingMachine();
6.
7.         // Idle -> Processing
8.         machine.pressButton();
9.         // Processing -> Dispensing
10.        machine.pressButton();
11.        // Dispensing -> Finished
12.        machine.pressButton();
13.        // Finished -> Idle
14.        machine.pressButton();
15.    }
16.}

```

ผลลัพธ์

Money inserted. Processing...

Dispensing item...

Item dispensed. Thank you!

Ready for next customer.

ตัวอย่างที่ 3 ระบบจัดการโหมดเครื่องปรับอากาศ (Air Conditioner Mode Control) เครื่องปรับอากาศสามารถมีโหมดต่าง ๆ เช่น

- Cooling หมายถึง โหมดทำความเย็น

- Heating หมายถึง โหมดทำความร้อน
- Fan หมายถึง โหมดพัดลม
- Off หมายถึง ปิดเครื่อง

```

1. // State interface
2. interface ACState {
3.     void pressButton(AirConditioner ac);
4. }

```

```

1. // Concrete State: Off
2. class OffState implements ACState {
3.
4.     @Override
5.     public void pressButton(AirConditioner ac) {
6.         System.out.println("AC is now in Cooling mode.");
7.         ac.setState(new CoolingState());
8.     }
9. }

```

```

1. // Concrete State: Cooling
2. class CoolingState implements ACState {
3.
4.     @Override
5.     public void pressButton(AirConditioner ac) {
6.         System.out.println("AC is now in Heating mode.");
7.         ac.setState(new HeatingState());
8.     }
9. }

```

```

1. // Concrete State: Heating
2. class HeatingState implements ACState {
3.
4.     @Override
5.     public void pressButton(AirConditioner ac) {
6.         System.out.println("AC is now in Fan mode.");
7.         ac.setState(new FanState());
8.     }
9. }

```

```

1. // Concrete State: Fan
2. class FanState implements ACState {
3.
4.     @Override
5.     public void pressButton(AirConditioner ac) {
6.         System.out.println("AC is now Off.");
7.         ac.setState(new OffState());
8.     }
9. }

```

```

1. // Context: AirConditioner
2. class AirConditioner {
3.
4.     private ACState state;
5.
6.     public AirConditioner() {
7.         // Initial state
8.         state = new OffState();
9.     }
10.
11. public void setState(ACState state) {
12.     this.state = state;
13. }
14.
15. public void pressButton() {
16.     state.pressButton(this);
17. }
18.}

1. // Main Class
2. public class ACStateExample {
3.     public static void main(String[] args) {
4.
5.         AirConditioner ac = new AirConditioner();
6.
7.         ac.pressButton(); // Off -> Cooling
8.         ac.pressButton(); // Cooling -> Heating
9.         ac.pressButton(); // Heating -> Fan
10.        ac.pressButton(); // Fan -> Off
11.    }
12.}

```

ผลลัพธ์

AC is now in Cooling mode.

AC is now in Heating mode.

AC is now in Fan mode.

AC is now Off.